

If you've been writing JS for any length of time, odds are the syntax is pretty familiar to you. There are certainly many quirks, but overall it's a fairly reasonable and straightforward syntax that draws many similarities from other languages.

However, ES6 adds quite a few new syntactic forms which are going to take some getting used to. In this chapter we'll tour through them to find out what's in store.



At the time of this writing, some of the features in this book have been implemented in various browsers (Firefox, Chrome, etc.), but many others have not, or the features are only partially implemented. Your experience may be mixed trying these examples directly. If so, try them out with transpilers, as most of these features are covered by those tools. ES6Fiddle (<http://www.es6fiddle.net/>) is a great, easy-to-use playground for trying out ES6, as is the online REPL for the Babel transpiler (<http://babeljs.io/repl/>).

Block-Scoped Declarations

You're probably aware that the fundamental unit of variable scoping in JavaScript has always been the function. If you needed to create a block of scope, the most prevalent way to do so was the IIFE (immediately invoked function expression), such as:

```
var a = 2;  
  
(function IIFE(){  
  var a = 3;  
  console.log( a );  // 3  
})();
```

```
console.log( a );      // 2
```

let Declarations

However, we can now create declarations which are bound to any block, called (unsurprisingly) *block scoping*. This means all we need is a pair of { .. } to create a scope. Instead of using `var`, which always declares variables attached to the enclosing function (or global, if top level) scope, use `let`:

```
var a = 2;

{
  let a = 3;
  console.log( a );  // 3
}

console.log( a );    // 2
```

It's not very common or idiomatic thus far in JS to use a standalone { .. } block as shown there, but it's always been totally valid. And developers from other languages that have *block scoping* will readily recognize that pattern.

I'm going to suggest that I think this is the far better way to create block-scoped variables, with a dedicated { .. } block. Moreover, I will also strongly suggest you should always put the `let` declaration(s) at the very top of that block. If you have more than one to declare, I'd recommend using just one `let`.

Stylistically, I even prefer to put the `let` on the same line as the opening {, to make it clearer that this block is only for the purpose of declaring the scope for those variables.

```
{  let a = 2, b, c;
  // ..
}
```

Now, that's going to look strange and it's not likely going to match the recommendations by most other ES6 literature. But I have reasons for my madness.

There's another proposed form of the `let` declaration called the `let-block`, which looks like:

```
let (a = 2, b, c) {
  // ..
}
```

That form is what I'd called *explicit* block scoping, whereas the `let ..` declaration form that mirrors `var` is more *implicit*, since it kind of hijacks whatever { .. } pair it's found in. Generally developers find *explicit* mechanisms a bit more preferable than *implicit* mechanisms, and I claim this is one of those cases.

If you compare the previous two snippet forms, they're very similar, and in my opinion both qualify stylistically as *explicit* block scoping. Unfortunately, the `let (..) { .. }` form, the most *explicit* of the options, was not adopted in ES6. That may be revisited post-ES6, but for now the former option is our best bet, I think.

To reinforce the *implicit* nature of `let ..` declarations, consider these usages:

```
let a = 2;

if (a > 1) {
  let b = a * 3;
  console.log( b );      // 6

  for (let i = a; i <= b; i++) {
    let j = i + 10
    console.log( j );
  }
  // 12 13 14 15 16

  let c = a + b;
  console.log( c );      // 8
}
```

Quick quiz without looking back at that snippet: which variable(s) exist only inside the `if` statement, and which variable(s) existing only inside the `for` loop?

The answers: the `if` statement contains `b` and `c` block-scoped variables, and the `for` loop contains `i` and `j` block-scoped variables.

Did you have to think about it for a moment? Does it surprise you that `i` isn't added to the enclosing `if` statement scope? That mental pause and questioning — I call it a “mental tax” — comes from the fact that this `let` mechanism is not only new to us, but it's also *implicit*.

There's also hazard in the `let c = ..` declaration appearing so far down in the scope. Unlike traditional `var`-declared variables, which are attached to the entire enclosing function scope regardless of where they appear, `let` declarations attach to the block scope but are not initialized until they appear in the block.

Accessing a `let`-declared variable earlier than its `let ..` declaration/initialization causes an error, whereas with `var` declarations the ordering doesn't matter (except stylistically).

Consider:

```
{
  console.log( a );  // undefined
  console.log( b );  // ReferenceError!

  var a;
```

```

    let b;
  }

```



This `ReferenceError` from accessing too-early `let`-declared references is technically called a *TDZ* (temporal dead zone) error — you’re accessing a variable that’s been declared but not yet initialized. This will not be the only time we see *TDZ* errors — they crop up in several places in ES6. Also, note that “initialized” doesn’t require explicitly assigning a value in your code, as `let b;` is totally valid. A variable that’s not given an assignment at declaration time is assumed to have been assigned the `undefined` value, so `let b;` is the same as `let b = undefined;`. Explicit assignment or not, you cannot access `b` until the `let b` statement is run.

One last gotcha: `typeof` behaves differently with *TDZ* variables than it does with undeclared (or declared!) variables.

```

{
  if (typeof a === "undefined") {
    console.log( "cool" );
  }

  if (typeof b === "undefined") {    // ReferenceError!
    // ..
  }

  // ..

  let b;
}

```

The `a` is not declared, so `typeof` is the only safe way to check for its existence or not. But `typeof b` throws the *TDZ* error because much farther down in the code there happens to be a `let b` declaration. Oops.

Now it should be clearer why I strongly prefer — no, I insist — `let` declarations must all be at the top of the scope. That totally avoids the accidental errors of accessing too early. It also makes it more *explicit* when you look at the start of a block, any block, what variables it contains.

Your blocks don’t have to share their original behavior with scoping behavior.

This explicitness on your part, which is up to you to maintain with discipline, will save you lots of refactor headaches and footguns down the line.



For more information on `let` and block scoping, see Chapter 3 of the “*Scope & Closures*” title of this series.

let + for

The only exception I’d make to the preference for the *explicit* form of `let` declaration blocking is a `let` that appears in the header of a `for` loop. The reason may seem nuanced, but I consider it to be one of the more important ES6 features.

Consider:

```
var funcs = [];  
  
for (let i = 0; i < 5; i++) {  
  funcs.push( function(){  
    console.log( i );  
  } );  
}  
  
funcs[3]();    // 3
```

The `let i` in the `for` header declares an `i` not just for the `for` loop itself, but it redeclares a new `i` for each iteration of the loop. That means that closures created inside the loop iteration close over those per-iteration variables the way you’d expect.

If you tried that same snippet but with `var i` in the `for` loop header, you’d get 5 instead of 3, because there’d only be one `i` in the outer scope that was closed over, instead of a new `i` for each iteration’s function to close over.

You could also have accomplished the same thing slightly more verbosely:

```
var funcs = [];  
  
for (var i = 0; i < 5; i++) {  
  let j = i;  
  funcs.push( function(){  
    console.log( j );  
  } );  
}  
  
funcs[3]();    // 3
```

Here, we forcibly create a new `j` for each iteration, and then the closure works the same way. I prefer the former approach; that extra special capability is why I endorse the `for (let .. ) ..` form. It could be argued it’s somewhat more *implicit*, but it’s *explicit* enough, and useful enough, for my tastes.